

CIS 25 day 4

functions, arrays, strings, style guide

CONST, AND WHY NO GLOBALS

the const keyword, and why non-const global variables are off limits

CONST CATCHES MISTAKES FOR YOU

- If you (or a teammate) accidentally try to reassign a const variable, the code won't compile.
- This turns a silent logic bug into a loud, immediate compiler error -- exactly when you want to find it.



```
const int MAX_SCORE = 100;  
...  
MAX_SCORE = 200; // oops
```

```
error: assignment of read-only  
variable 'MAX_SCORE'  
    MAX_SCORE = 200;  
    ~~~~~^~~~~
```

the compiler refuses to build this program

STYLE: NAMING CONSTANTS

- This connects to a rule from the style guide: names of constants should be in ALL_CAPS.
- No "magic numbers." If you use a number repeatedly (or even just once, if it needs explaining), give it a name.
- Use `const int ARRAY_SIZE = 4;` instead of scattering the number 4 through your code.

```
// unclear -- what is 4? what is 0.15?  
double tax = price * 0.15;  
  
// clear -- self-documenting  
const double TAX_RATE = 0.15;  
double tax = price * TAX_RATE;
```

WHY NO NON-CONST
GLOBAL VARIABLES?

PROBLEM: NAMESPACE POLLUTION

- A global variable's name is visible to every file and function in your program.
- In a big program (or one built from several files), it's easy for two globals to accidentally collide -- same name, different intended meaning.
- Local variables don't have this problem: two functions can each safely use a variable named `count`.



```
// physics.cpp
double count; // particles

// grading.cpp
int count; // # of students

// linker error: 'count'
// defined more than once
```

same name, two different meanings -- collision

PROBLEM: TESTABILITY

- A function that reads or writes a global has hidden inputs and outputs -- its behavior depends on state that isn't in its parameter list.
- To test it, you can't just call it with inputs and check the output. You first have to set up the right global state, and that setup can leak between tests.
- A function that only uses its parameters and local variables is easy to test in isolation.



```
int total; // global

void addToTotal(int n) {
    total += n;
}

// to test this, you must first
// set 'total' correctly yourself
```

hidden dependency on state outside the function

PROBLEM: DEBUGGING

- A global can be changed by any function, anywhere in the program, at any time.
- When it holds a wrong value, you have no way to narrow down where -- you have to search the entire codebase.
- A local variable can only be changed inside the one function (or block) it lives in, so a bug is much easier to trace.

```
int score; // global

void f() { score++; }
void g() { score = 0; }
void h() { score -= 5; }

// score is wrong. which one
// of these did it? all of
// them can reach it.
```

any function can reach in and change it -- no way to narrow the search

AGENDA

- Arrays & two-dimensional arrays
- More recursion! More! More!
 - (mostly this is just to introduce you to the call stack, but I'll show a pretty fun technique too)
- String functions
- Style guide

PASS BY REFERENCE VS. PASS BY VALUE

PASS BY VALUE

Consider this relatively straightforward program. The function `sumThree` takes in three integers, then returns their sum as an integer:

```
1 #include <iostream>
2
3 using namespace std;
4
5 // returns the sum of three numbers
6 int sumThree(int a, int b, int c){
7     return a + b + c;
8 }
9
10 int main(){
11     int firstInt = 2;
12     int secondInt = 3;
13     int thirdInt = 5;
14
15     cout << "The sum of the numbers is: "
16         << sumThree(firstInt, secondInt, thirdInt) << endl;
17 }
18 }
```



one-line function that just returns a sum

PASS BY VALUE

When `sumThree` is called (on line 17), the *values* of the three variables given as arguments are copied into the parameters of `sumThree`

So when we hit line 17, we call `sumThree`. When we call `sumThree`, three new ints (named `a`, `b`, and `c`) are declared, and their values are set equal to the values of the variables given as arguments.

```
1 #include <iostream>
2
3 using namespace std;
4
5 // returns the sum of three numbers
6 int sumThree(int a, int b, int c){
7     return a + b + c;
8 }
9
10 int main(){
11     int firstInt = 2;
12     int secondInt = 3;
13     int thirdInt = 5;
14
15
16     cout << "The sum of the numbers is: "
17         << sumThree(firstInt, secondInt, thirdInt) << endl;
18 }
```

PASS BY VALUE

When `sumThree` is called (on line 19), the *values* of the three variables given as arguments are copied into the parameters of `sumThree`:

main:

`firstInt`



2

`secondInt`



3

`thirdInt`



5

`sumThree:`

`a`



2

`b`



3

`c`



5

`a`, `b`, and `c` are totally new variables, that are set equal to the values of `firstInt`, `secondInt`, and `thirdInt` when the `sumThree` function is called.

PASS BY REFERENCE

Now consider this version of the program:

it works exactly the same... but we pass in each of the variables *by reference*. This means that instead of *copying the value* stored in a particular variable, we *give the address of that variable*.

What's the difference?

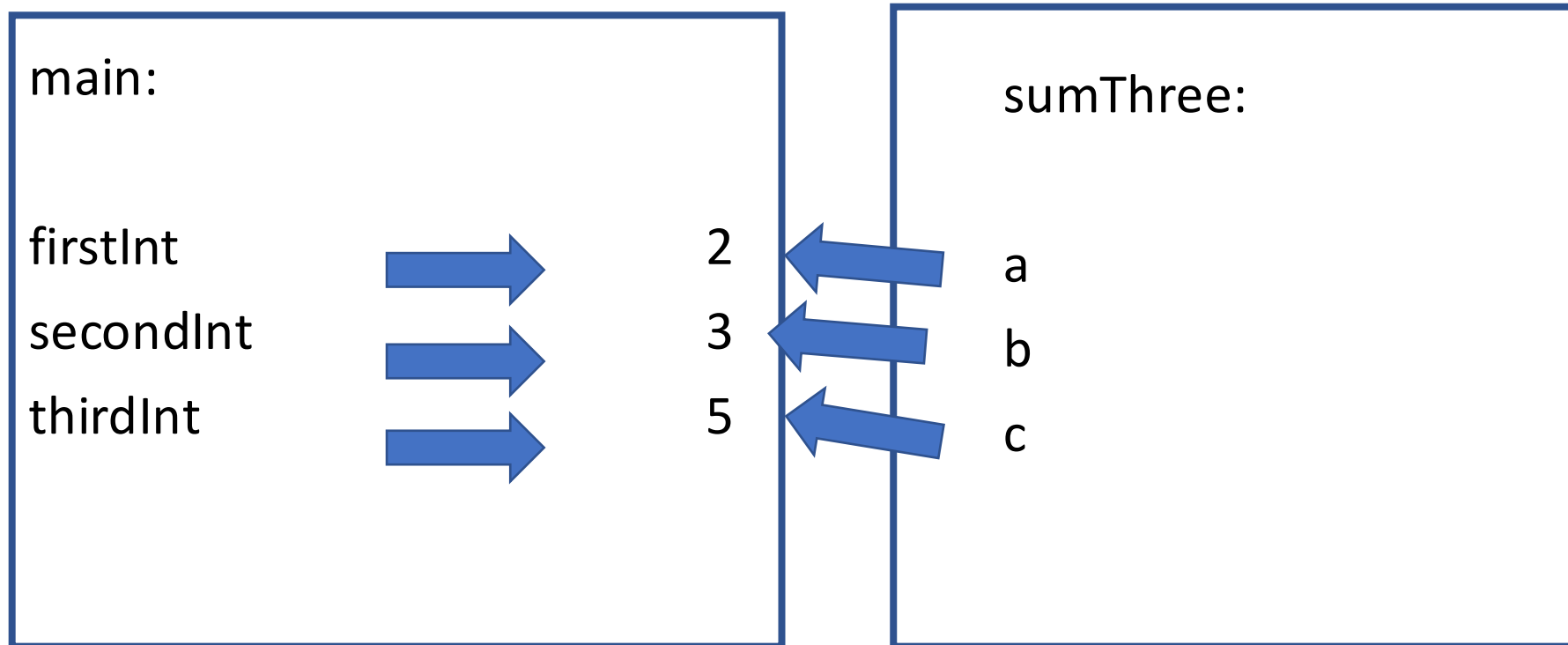
You indicate that a variable is passed by reference by putting a & sign before the variable name when specifying the parameters:



```
10 int sumThreeByReference(int &a, int &b, int &c){
11     return a+b+c;
12 }
13
14
15 int main(){
16     int firstInt = 2;
17     int secondInt = 3;
18     int thirdInt = 5;
19
20
21     cout << "The sum of the numbers is: "
22         << sumThreeByReference(firstInt, secondInt, thirdInt) << endl;
23 }
```

PASS BY REFERENCE

When `sumThreeByReference` is called on line 22, this time no new variables are made. 'a' in `sumThreeByReference` is *literally the same variable* as `firstInt` in `main`:



a, b, and c are not copies of `firstInt`, `secondInt`, and `thirdInt`. They are instead new names referring to `firstInt`, `secondInt`, and `thirdInt`.

PASS BY VALUE

What difference does it make? Let's try changing the values of variables in sumThree:

main:

firstInt	→	2
secondInt	→	3
thirdInt	→	5

sumThree:

a	→	2
b	→	3
c	→	5

a, b, and c are totally new variables, that are set equal to the values of firstInt, secondInt, and thirdInt when the sumThree function is called.

PASS BY VALUE

What difference does it make? Let's try changing the values of variables in sumThree:

```
5 // returns the sum of three numbers
6 int sumThree(int a, int b, int c){
7     a += 1;
8     return a + b + c;
9 }
10
11 // returns the sum of three numbers. all parameters passed by reference
12 int sumThreeByReference(int &a, int &b, int &c){
13     return a+b+c;
14 }
15
16
17 int main(){
18     int firstInt = 2;
19     int secondInt = 3;
20     int thirdInt = 5;
21
22     sumThree(firstInt, secondInt, thirdInt);
23
24     cout << "firstInt after SumThree: " << firstInt << endl;
25
26     sumThreeByReference(firstInt, secondInt, thirdInt);
27
28     cout << "firstInt after SumThreeByReference: " << firstInt << endl;
```

result: as expected, firstInt is unchanged



```
Bens-MacBook-Air-2:code-samples benallen$ ./sumThree
firstInt after SumThree: 2
```

PASS BY REFERENCE

What difference does it make? Let's try changing the values of variables in sumThreeByReference:

```
11 // returns the sum of three numbers. all parameters passed by reference
12 int sumThreeByReference(int &a, int &b, int &c){
13     a += 1;
14     return a+b+c;
15 }
16
17
18 int main(){
19     int firstInt = 2;
20     int secondInt = 3;
21     int thirdInt = 5;
22
23     sumThree(firstInt, secondInt, thirdInt);
24
25     cout << "firstInt after SumThree: " << firstInt << endl;
26
27     sumThreeByReference(firstInt, secondInt, thirdInt);
28
29     cout << "firstInt after SumThreeByReference: " << firstInt << endl;
```

result: after
sumThreeByReference, firstInt
is increased by 1

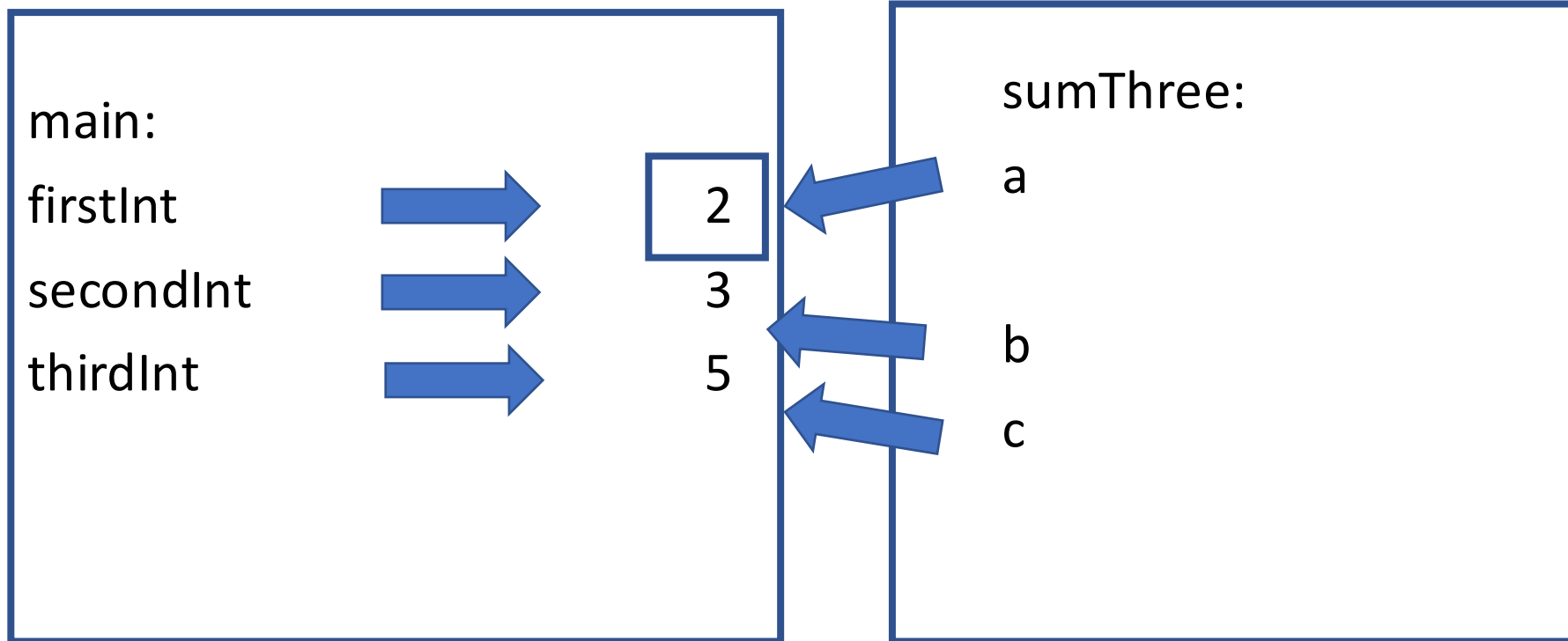


```
Bens-MacBook-Air-2:code-samples allen$ ./sumThree
firstInt after SumThree: 2
firstInt after SumThreeByReference: 3
Bens-MacBook-Air-2:code-samples allen$
```

PASS BY REFERENCE

This is because `firstInt` and `a` are *literally the same variable*.

Both `firstInt` and `a` are exactly the same thing, just by different names



`a`, `b`, and `c` are not copies of `firstInt`, `secondInt`, and `thirdInt`. They are instead new names referring to `firstInt`, `secondInt`, and `thirdInt`.

WHY PASS BY REFERENCE MATTERS #1

- Arrays are *always* pass-by-reference. Remember that if you change an array in a function, it changes it *everywhere*

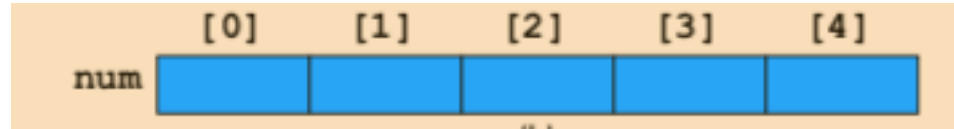
this produces
the following
output:

```
Bens-MacBook-Air-2:code-samples benallen$ ./arrayChanger
myArray before arrayChanger:
0 1 2 3 4 5 6 7 8 9
myArray before arrayChanger:
137 1 2 3 4 5 6 7 8 9
```

```
6
7 void arrayChanger(int theArray[], int arrayLength){
8     if (arrayLength > 0){
9         theArray[0] = 137;
10    }
11 }
12
13 int main(){
14     int myArray[LENGTH];
15     for (int index = 0; index < LENGTH; index++){
16         myArray[index] = index;
17     }
18     cout << "myArray before arrayChanger: " << endl;
19     for (int index = 0; index < LENGTH; index++){
20         cout << myArray[index] << " ";
21     }
22     cout << endl;
23
24     arrayChanger(myArray, LENGTH);
25
26     cout << "myArray after arrayChanger: " << endl;
27     for (int index = 0; index < LENGTH; index++){
28         cout << myArray[index] << " ";
29     }
30     cout << endl;
31 }
```

ARRAYS[]

ARRAYS



- So let's (formally) talk about arrays.
- Declare an array by either giving the number stored in the array in brackets:

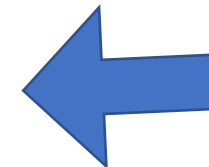
```
int myArray[50];    // declares a 50 element array of ints
```

- Or by explicitly initializing the array:

```
int evensArray[] = {2, 4, 6, 8, 10};
```

- If you want, you can *partially* initialize the array:

```
double partialArray[10] = {2.0, 5.0, 7.0}
```



the first three elements in
this array are initialized.
The next seven aren't.

ARRAYS

- An array of length n that stores variables of type t is in memory a contiguous block of $n * (\text{size of one variable of type } t)$ bytes.
- For an array named `num`, `num[0]` is the value stored at the address of the (size of t) bytes of the array in memory.
- Copying an array into another array must be done *component by component*. To copy a 5-element array, you must do something like this.

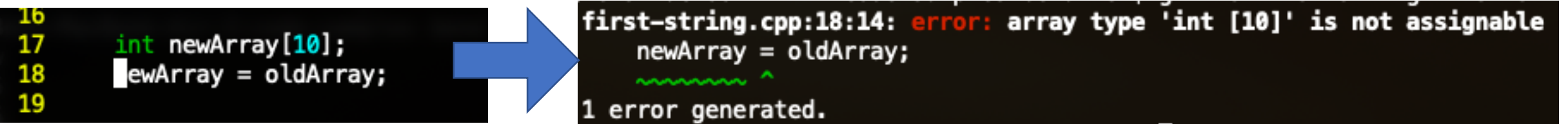
```
for (int index = 0; index < 5; index++){  
    newArray[index] = oldArray[index];  
}
```

ARRAYS

- An array of length n that stores variables of type t is in memory a contiguous block of $n * (\text{size of one variable of type } t)$ bytes.
- For an array named `num`, `num[0]` is the value stored at the address of the (size of t) bytes of the array in memory.
- Copying an array into another array must be done *component by component*. To copy a 5-element array, you must do something like this.

```
for (int index = 0; index < 5; index++){  
    newArray[index] = oldArray[index];  
}
```

- If you try this you will get an error:
`newArray = oldArray;`



```
16  
17 int newArray[10];  
18 newArray = oldArray;  
19
```

first-string.cpp:18:14: error: array type 'int [10]' is not assignable
newArray = oldArray;
~~~~~ ^  
1 error generated.



# CONSTRAINTS ON ARRAYS

- Functions cannot return arrays
- Arrays are always (effectively) pass-by-reference – when you pass an array to a function, you are passing the address of the first element in the array.
- Officially in C++ you *cannot* use variables as sizes in array declarations; i.e.  

```
int size = 10;  
int myArray[size];
```

is not standard C++. Some compilers (most notably gcc) allow this as an extension. Please do not do this in your assignments – we want to whenever possible stick to standard C++.
- (you will not often hear me badmouthing gcc. but even so, I think it was a bad idea for them to include this).
- You CAN use a constant to set array size: i.e.  

```
const int ARRAY_SIZE = 4;  
int myArray[ARRAY_SIZE];
```

is perfectly fine

# String objects

and how to manipulate them

# SOME STRING FUNCTIONS

(See page  
496 of  
textbook)

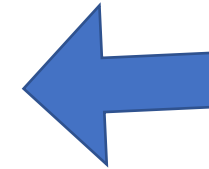
| Expression                        | Effect                                                                                                                                                           |
|-----------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>strVar.at(index)</code>     | Returns the element at the position specified by <code>index</code> .                                                                                            |
| <code>strVar[index]</code>        | Returns the element at the position specified by <code>index</code> .                                                                                            |
| <code>strVar.append(n, ch)</code> | Appends <code>n</code> copies of <code>ch</code> to <code>strVar</code> , where <code>ch</code> is a <code>char</code> variable or a <code>char</code> constant. |
| <code>strVar.append(str)</code>   | Appends <code>str</code> to <code>strVar</code> .                                                                                                                |
| <code>strVar.clear()</code>       | Deletes all the characters in <code>strVar</code> .                                                                                                              |
| <code>strVar.compare(str)</code>  | Returns 1 if <code>strVar &gt; str</code> ; returns 0 if <code>strVar == str</code> ; returns -1 if <code>strVar &lt; str</code> .                               |
| <code>strVar.empty()</code>       | Returns <code>true</code> if <code>strVar</code> is empty; otherwise it returns <code>false</code> .                                                             |
| <code>strVar.erase()</code>       | Deletes all the characters in <code>strVar</code> .                                                                                                              |
| <code>strVar.erase(pos, n)</code> | Deletes <code>n</code> characters from <code>strVar</code> starting at position <code>pos</code> .                                                               |

apologies for  
how ugly /boring these  
slides are...

# SOME STRING FUNCTIONS

(See page  
496 of  
textbook)

| Expression                        | Effect                                                                                                                                                           |
|-----------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>strVar.at(index)</code>     | Returns the element at the position specified by <code>index</code> .                                                                                            |
| <code>strVar[index]</code>        | Returns the element at the position specified by <code>index</code> .                                                                                            |
| <code>strVar.append(n, ch)</code> | Appends <code>n</code> copies of <code>ch</code> to <code>strVar</code> , where <code>ch</code> is a <code>char</code> variable or a <code>char</code> constant. |
| <code>strVar.append(str)</code>   | Appends <code>str</code> to <code>strVar</code> .                                                                                                                |
| <code>strVar.clear()</code>       | Deletes all the characters in <code>strVar</code> .                                                                                                              |
| <code>strVar.compare(str)</code>  | Returns 1 if <code>strVar &gt; str</code> ; returns 0 if <code>strVar == str</code> ; returns -1 if <code>strVar &lt; str</code> .                               |
| <code>strVar.empty()</code>       | Returns <code>true</code> if <code>strVar</code> is empty; otherwise it returns <code>false</code> .                                                             |
| <code>strVar.erase()</code>       | Deletes all the characters in <code>strVar</code> .                                                                                                              |
| <code>strVar.erase(pos, n)</code> | Deletes <code>n</code> characters from <code>strVar</code> starting at position <code>pos</code> .                                                               |

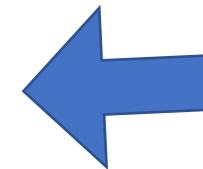


note that these two  
are equivalent: the  
array subscript  
notation is included to  
maintain compatibility  
with C

# SOME STRING FUNCTIONS

(See page  
496 of  
textbook)

| Expression                        | Effect                                                                                                                                                           |
|-----------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>strVar.at(index)</code>     | Returns the element at the position specified by <code>index</code> .                                                                                            |
| <code>strVar[index]</code>        | Returns the element at the position specified by <code>index</code> .                                                                                            |
| <code>strVar.append(n, ch)</code> | Appends <code>n</code> copies of <code>ch</code> to <code>strVar</code> , where <code>ch</code> is a <code>char</code> variable or a <code>char</code> constant. |
| <code>strVar.append(str)</code>   | Appends <code>str</code> to <code>strVar</code> .                                                                                                                |
| <code>strVar.clear()</code>       | Deletes all the characters in <code>strVar</code> .                                                                                                              |
| <code>strVar.compare(str)</code>  | Returns 1 if <code>strVar &gt; str</code> ; returns 0 if <code>strVar == str</code> ; returns -1 if <code>strVar &lt; str</code> .                               |
| <code>strVar.empty()</code>       | Returns <code>true</code> if <code>strVar</code> is empty; otherwise it returns <code>false</code> .                                                             |
| <code>strVar.erase()</code>       | Deletes all the characters in <code>strVar</code> .                                                                                                              |
| <code>strVar.erase(pos, n)</code> | Deletes <code>n</code> characters from <code>strVar</code> starting at position <code>pos</code> .                                                               |

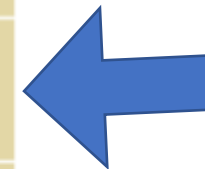


you can place  
either chars or  
other strings at  
the end of a  
string

# SOME STRING FUNCTIONS

(See page  
496 of  
textbook)

| Expression                        | Effect                                                                                                                                                           |
|-----------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>strVar.at(index)</code>     | Returns the element at the position specified by <code>index</code> .                                                                                            |
| <code>strVar[index]</code>        | Returns the element at the position specified by <code>index</code> .                                                                                            |
| <code>strVar.append(n, ch)</code> | Appends <code>n</code> copies of <code>ch</code> to <code>strVar</code> , where <code>ch</code> is a <code>char</code> variable or a <code>char</code> constant. |
| <code>strVar.append(str)</code>   | Appends <code>str</code> to <code>strVar</code> .                                                                                                                |
| <code>strVar.clear()</code>       | Deletes all the characters in <code>strVar</code> .                                                                                                              |
| <code>strVar.compare(str)</code>  | Returns 1 if <code>strVar &gt; str</code> ; returns 0 if <code>strVar == str</code> ; returns -1 if <code>strVar &lt; str</code> .                               |
| <code>strVar.empty()</code>       | Returns <code>true</code> if <code>strVar</code> is empty; otherwise it returns <code>false</code> .                                                             |
| <code>strVar.erase()</code>       | Deletes all the characters in <code>strVar</code> .                                                                                                              |
| <code>strVar.erase(pos, n)</code> | Deletes <code>n</code> characters from <code>strVar</code> starting at position <code>pos</code> .                                                               |



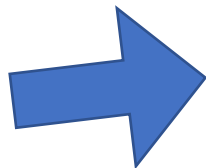
VERY USEFUL

# MORE STRING FUNCTIONS

(See page  
497 of  
textbook)

potential pitfall: technically, `str.length()` returns a variable of type `size_t`. it is best practice when printing a `size_t` to cast it to an `int`: i.e.,

```
cout << static_cast<unsigned int> strVar.length();
```



| Expression                                      | Effect                                                                                                                                                                                                                                                                                  |
|-------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>strVar.find(str)</code>                   | Returns the index of the first occurrence of <code>str</code> in <code>strVar</code> . If <code>str</code> is not found, the special value <code>string::npos</code> is returned.                                                                                                       |
| <code>strVar.find(str, pos);</code>             | Returns the index of the first occurrence at or after <code>pos</code> where <code>str</code> is found in <code>strVar</code> .                                                                                                                                                         |
| <code>strVar.find_first_of(str, pos)</code>     | Returns the index of the first occurrence of any character of <code>strVar</code> in <code>str</code> . The search starts at <code>pos</code> .                                                                                                                                         |
| <code>strVar.find_first_not_of(str, pos)</code> | Returns the index of the first occurrence of any character of <code>str</code> not in <code>strVar</code> . The search starts at <code>pos</code> .                                                                                                                                     |
| <code>strVar.insert(pos, n, ch);</code>         | Inserts <code>n</code> occurrences of the character <code>ch</code> at index <code>pos</code> into <code>strVar</code> ; <code>pos</code> and <code>n</code> are of type <code>string::size_type</code> ; and <code>ch</code> is a character.                                           |
| <code>strVar.insert(pos, str);</code>           | Inserts all the characters of <code>str</code> at index <code>pos</code> into <code>strVar</code> .                                                                                                                                                                                     |
| <code>strVar.length()</code>                    | Returns a value of type <code>string::size_type</code> giving the number of characters in <code>strVar</code> .                                                                                                                                                                         |
| <code>strVar.replace(pos, n, str);</code>       | Starting at index <code>pos</code> , replaces the next <code>n</code> characters of <code>strVar</code> with all the characters of <code>str</code> . If <code>n &gt; length</code> of <code>strVar</code> , then all the characters until the end of <code>strVar</code> are replaced. |
| <code>strVar.substr(pos, len)</code>            | Returns a string which is a substring of <code>strVar</code> starting at <code>pos</code> . The length of the substring is at most <code>len</code> characters. If <code>len</code> is too large, it means "to the end" of the string in <code>strVar</code> .                          |
| <code>strVar.size()</code>                      | Returns a value of type <code>string::size_type</code> giving the number of characters in <code>strVar</code> .                                                                                                                                                                         |
| <code>strVar.swap(str1);</code>                 | Swaps the contents of <code>strVar</code> and <code>str1</code> . <code>str1</code> is a <code>string</code> variable.                                                                                                                                                                  |

(for anyone who's had to work with regular c-strings (arrays of chars), you'll really appreciate every last one of these functions.)

# style guide

please follow this, for your sake and for mine